

OPERATING SYSTEMS — TOP 100 MOST IMPORTANT TOPICS

For: College Exams | Placement Preparation | Technical Interviews | Quick Revision

SECTION 1 — OS BASICS

1. Operating System Basics

Definition

An **Operating System** is system software that acts as an intermediary between the user and hardware, managing resources and providing services for programs.

Key Points

- **Resource Manager** → Manages CPU, memory, I/O, and storage.
- **Extended Machine** → Hides hardware complexity from users.
- **Program Executor** → Provides environment to run user programs.

In One Line

OS is the master program that manages hardware and enables other programs to run.

2. Functions of Operating System

Key Functions

Function	Description
Process Management	Create, schedule, and terminate processes

Memory Management	Allocate/deallocate memory to processes
File-System Management	Manage files, directories, and storage
I/O Management	Control I/O devices and drivers
Security & Protection	Prevent unauthorized access
Networking	Support communication between systems
Error Detection	Detect and handle hardware/software errors
Accounting	Track resource usage per user/process

In One Line

OS functions span from process and memory management to I/O, security, and networking.

3. Types of Operating Systems

Type	Key Feature	Example
Batch OS	Jobs collected and executed without interaction	IBM OS/360
Multiprogramming OS	Multiple jobs in memory; CPU switches to reduce idle time	Early UNIX

Multitasking OS	Fast CPU switching; illusion of parallel execution	UNIX, Linux
Time-sharing OS	CPU divided into time slices; one slice per user	MULTICS
Real-time OS	Strict time deadline guarantees	VxWorks
Distributed OS	Multiple CPUs appear as one system	Amoeba
Network OS	Machines share resources over a network	Novell NetWare
Embedded OS	Designed for specific devices; limited resources	FreeRTOS
Mobile OS	Touch, battery, and sensor optimized	Android, iOS

In One Line

OS types reflect the execution model, hardware environment, and user interaction mode.

4. OS Services

Services Provided

- **Program Execution** → Load and run programs; report errors.
- **I/O Operations** → Manage device access for programs.
- **File-System Manipulation** → Create, read, write, delete files.

- **Communication** → Process-to-process via shared memory or message passing.
- **Error Detection** → Detect CPU, memory, I/O errors.
- **Resource Allocation** → Allocate CPU, memory, I/O among processes.
- **Accounting** → Track resource usage.
- **Protection & Security** → Control access; prevent unauthorized use.

In One Line

OS services are what it provides to make computing safe, correct, and efficient.

5. Kernel

Definition

The **kernel** is the **core part of the OS** that has full control over hardware. It always runs in memory and manages processes, memory, I/O, and security.

Key Points

- Runs in **kernel mode** (privileged, full hardware access).
- Loaded into a **protected area** of memory.
- Provides **system calls** as the interface to user programs.
- Types: Monolithic, Microkernel, Modular, Hybrid.

In One Line

The kernel is the always-running core of the OS with direct hardware control.

6. User Mode vs Kernel Mode

Definition

Dual-mode operation allows the OS to protect itself and system components from faulty user programs by providing two modes of execution.

Comparison

Feature	User Mode	Kernel Mode
Privilege	Restricted	Full (privileged)
Hardware Access	No direct access	Direct access
Mode Bit	1	0
Used By	User applications	OS kernel
Crash Impact	Only that process	Entire system
Switch Trigger	System call / interrupt	Completion of service

How Mode Switches

```

User Program
  ↓ system call / interrupt (mode bit → 0)
Kernel Mode (OS handles request)
  ↓ return (mode bit → 1)
User Mode resumes

```

In One Line

Dual-mode (user/kernel) protects the OS from user programs using a hardware mode bit.

7. OS Structures

Structure	Description	Pros	Cons
Monolithic	All OS services in one large kernel	Fast	Hard to maintain

Layered	OS divided into layers; each built on lower	Easy to debug	Poor performance
Microkernel	Minimal kernel; other services in user space	Secure, reliable	Slow (message passing)
Modular	Core kernel + dynamically loadable modules	Flexible, efficient	Slightly complex
Hybrid	Mix of monolithic + microkernel	Best of both	Complex

In One Line

OS structure defines how kernel services are organized; modern OSes use modular or hybrid structures.

8. Monolithic vs Microkernel

Feature	Monolithic Kernel	Microkernel
Services in kernel	All	Only IPC, memory, scheduling
Speed	Fast	Slower (IPC overhead)
Size	Large	Small
Reliability	Lower (one crash = system crash)	Higher (services in user space)

Extensibility	Harder	Easier
Example	Linux, UNIX	Mach, MINIX, L4

In One Line

Monolithic is fast but bulky; microkernel is modular and reliable but slower due to IPC.

9. System Calls

Definition

A **system call** is a programmatic way for a user application to request a **privileged service** from the OS kernel.

How It Works

```
User Program → calls library function (e.g., read())
↓
Library executes TRAP instruction (mode bit → 0)
↓
Kernel handles system call
↓
Returns result → mode bit → 1 → User mode resumes
```

Types of System Calls

Type	Examples
Process Control	<code>fork()</code> , <code>exec()</code> , <code>exit()</code> , <code>wait()</code>
File Management	<code>open()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code>

Device Management	<code>ioctl()</code> , <code>read()</code> , <code>write()</code>
Information Maintenance	<code>getpid()</code> , <code>time()</code> , <code>alarm()</code>
Communication	<code>pipe()</code> , <code>socket()</code> , <code>shmget()</code>
Protection	<code>chmod()</code> , <code>chown()</code> , <code>umask()</code>

In One Line

System calls are the controlled entry points into the kernel, enabling user programs to request privileged services.

10. Interrupts

Definition

An **interrupt** is a signal sent to the CPU to indicate an event that requires **immediate attention**, causing it to stop current execution and run an interrupt handler.

Types

- **Hardware Interrupt** → Generated by hardware devices (disk I/O complete, keyboard press).
- **Software Interrupt (Trap)** → Generated by programs (system call, divide by zero).
- **Exception** → Caused by errors (page fault, segmentation fault).

How It Works

```
Device/Program generates interrupt
↓
CPU saves current state (registers, PC → stack)
↓
CPU jumps to Interrupt Handler (ISR)
↓
```

ISR handles the interrupt

↓

CPU restores saved state and resumes

Key Terms

- **ISR (Interrupt Service Routine)** → Code that handles the interrupt.
- **Interrupt Vector Table** → Table of addresses of all ISRs.
- **Maskable** → Can be disabled (most hardware interrupts).
- **Non-Maskable (NMI)** → Cannot be disabled (critical hardware failures).

In One Line

Interrupts allow the CPU to respond to asynchronous events by saving state, running a handler, and restoring state.

...

SECTION 2 — PROCESS MANAGEMENT

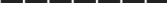
11. Process Concept

Definition

A **process** is a **program in execution**. It is an active entity that includes the program code, current activity, stack, heap, and data.

Components of a Process

Process in Memory:

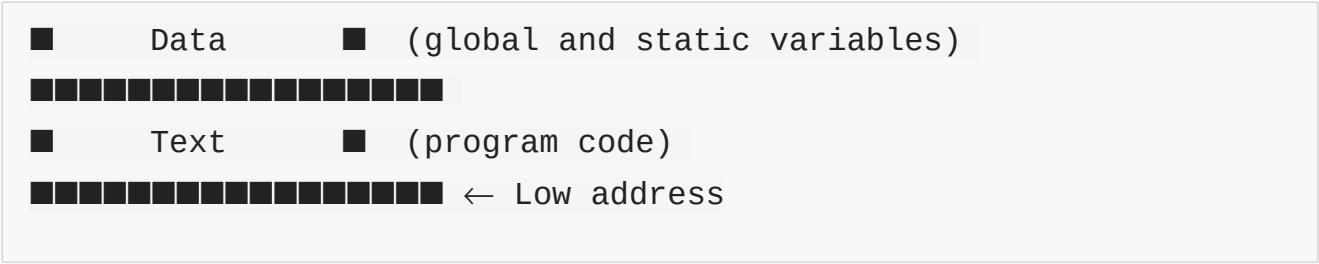
 ← High address

- Stack ■ (function calls, local variables)

■■■■■■■■■■

- Heap ■ (dynamic memory allocation)

■■■■■■■■■■



In One Line

A process = program code + current state + memory (stack, heap, data).

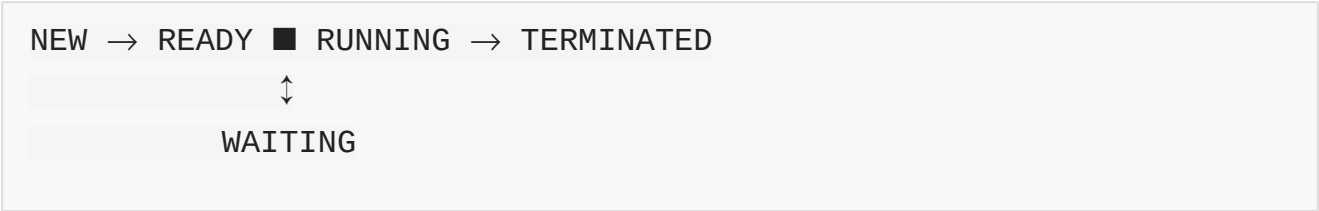
12. Program vs Process

Feature	Program	Process
Nature	Passive (stored on disk)	Active (running in memory)
Existence	Permanent	Temporary
Resources	None needed	Needs CPU, memory, files
Multiple instances	One copy on disk	Many processes from one program
Stored in	Secondary storage	Main memory

In One Line

A program is a passive file; a process is a running instance of that program.

13. Process States

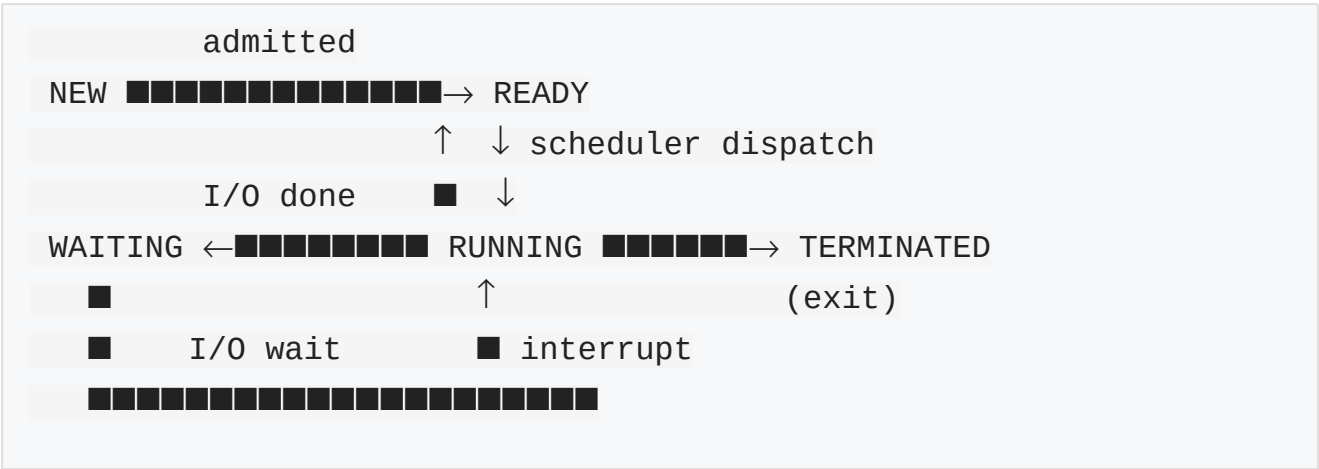


State	Description
New	Process is being created
Ready	Waiting for CPU; in Ready queue
Running	Instructions are being executed on CPU
Waiting	Waiting for I/O or an event
Terminated	Process has finished execution

In One Line

A process transitions through New → Ready → Running → Waiting → Terminated states.

14. Process State Diagram



Transitions

Transition	Cause
New → Ready	Process admitted to memory
Ready → Running	CPU scheduler dispatches process
Running → Waiting	Process waits for I/O or event
Waiting → Ready	I/O or event completes
Running → Ready	Interrupt / time-out (preemption)
Running → Terminated	Process calls <code>exit()</code>

In One Line

The state diagram shows all valid transitions a process can make during its lifetime.

15. PCB (Process Control Block)

Definition

A **PCB** is a data structure maintained by the OS for **each process**, storing all information needed to manage it.

PCB Contents

Field	Description
Process ID (PID)	Unique identifier
Process State	Current state (Ready, Running, etc.)

Program Counter	Address of next instruction to execute
CPU Registers	All register values saved on context switch
Scheduling Info	Priority, scheduling queue pointers
Memory Info	Page table, base/limit registers
I/O Status	Open files, I/O devices allocated
Accounting Info	CPU time, time limits

In One Line

PCB is the OS's data structure that stores all information about a process.

16. Process Creation

How a Process is Created

1. OS assigns a unique **PID**.
2. Allocates memory (code, data, stack, heap).
3. Initializes PCB.
4. Places process in the **Ready queue**.

Parent–Child Relationship

- Creating process = **parent**; created process = **child**.
- Forms a **process tree** (hierarchical).
- In UNIX: first process = `init` (PID=1), ancestor of all.

Resource Sharing Options (UNIX)

- Child shares **all resources** of parent.
- Child shares a **subset** of parent's resources.
- Child shares **no resources**; gets its own.

In One Line

Process creation assigns a PID, allocates memory, initializes PCB, and places the process in the Ready queue.

17. Process Termination

How a Process Terminates

- Process calls `exit()` → OS deallocates all resources.
- Parent can terminate a child using `kill()` or `abort()`.
- **Cascading Termination** → If parent terminates, OS terminates all its children.

After Termination

- PCB is kept until the **parent calls `wait()`** to collect exit status.
- If parent never calls `wait()` → **Zombie Process**.
- If parent terminates before child → **Orphan Process**.

In One Line

Process termination releases all resources; PCB remains until parent collects exit status via `wait()`.

18. `fork()`, `exec()`, `wait()`, `exit()`

Call	Description	Return Value
------	-------------	--------------

<code>fork()</code>	Creates a new child process (exact copy of parent)	Parent gets child's PID; child gets 0
<code>exec()</code>	Replaces current process image with a new program	Does not return on success
<code>wait()</code>	Parent blocks until a child terminates	PID of terminated child
<code>exit()</code>	Terminates the calling process	No return

Usage Flow

```

Parent calls fork()
  ↓
Child created (copy of parent)
  ↓
Child calls exec() → runs new program
Parent calls wait() → waits for child
  ↓
Child calls exit() → terminates
Parent unblocks → reads exit status

```

In One Line

`fork()` creates, `exec()` replaces, `wait()` synchronizes, and `exit()` terminates — the four pillars of UNIX process lifecycle.

19. Zombie Process

Definition

A **zombie process** is a process that has **completed execution** but whose **PCB still exists** because the parent has not yet called `wait()` to read its exit status.

Key Points

- Zombie takes up an **entry in the process table** (not memory/CPU).
- Too many zombies = **process table exhaustion** → cannot create new processes.
- **Prevention:** Parent must call `wait()` after child terminates.
- **Fix:** If parent never calls `wait()`, send `SIGCHLD` to parent or kill parent (orphan → adopted by `init`).

In One Line

A zombie is a finished process whose PCB remains because the parent hasn't called `wait()`.

20. Orphan Process

Definition

An **orphan process** is a process whose **parent has terminated** before the child has finished executing.

Key Points

- In UNIX/Linux, orphans are **adopted by `init` (PID=1)** or `systemd`.
- `init` periodically calls `wait()` → prevents orphans from becoming zombies.
- In Windows: orphan processes continue running independently.

Zombie vs Orphan

Feature	Zombie	Orphan
Process status	Terminated	Still running

Parent status	Alive, not called wait()	Terminated
PCB present	Yes	Yes
Solution	Parent calls wait()	Adopted by init

In One Line

An orphan is a running process whose parent has died; it gets adopted by init in UNIX.

SECTION 3 — CPU SCHEDULING

21. Process Scheduling

Definition

Process scheduling is the activity of selecting which process from the Ready queue gets the CPU next, maximizing CPU utilization.

Scheduling Queues

- **Job Queue** → All processes in the system.
- **Ready Queue** → Processes in memory, ready to run.
- **Device Queue** → Processes waiting for a specific I/O device.

In One Line

Scheduling decides which process gets the CPU; processes move through job, ready, and device queues.

22. Long-Term, Short-Term & Medium-Term Scheduler

Scheduler	Also Called	Job	Frequency
Long-Term	Job Scheduler	Selects from job pool → brings into memory (Ready queue)	Infrequent (seconds to minutes)
Short-Term	CPU Scheduler	Selects from Ready queue → gives CPU	Very frequent (milliseconds)
Medium-Term	Swapper	Swaps processes in/out of memory to reduce multiprogramming degree	Moderate

Key Points

- Long-term controls the **degree of multiprogramming**.
- Short-term must be **very fast** (runs every few ms).
- Medium-term performs **swapping** (removes partially-run processes from memory).

In One Line

Long-term admits, short-term dispatches, medium-term swaps — three levels of scheduling.

23. Dispatcher

Definition

The **dispatcher** is the OS module that gives **actual CPU control** to the process selected by the short-term scheduler.

Dispatcher Responsibilities

1. **Context switching** → Save state of old process; restore state of new process.
2. **Switch to user mode** → Change mode bit from kernel (0) to user (1).
3. **Jump to correct location** → Resume the new process at its last PC value.

Dispatch Latency

- Time taken by dispatcher to stop one process and start another.
- Should be as **small as possible** (pure overhead).

In One Line

The dispatcher performs the actual switch from one process to another — context switch + mode switch + jump.

24. Context Switching

Definition

Context switching is the process of **saving the state of the current process** (into its PCB) and **restoring the state of the next process** (from its PCB) when the CPU switches between processes.

What is Saved/Restored

- Program Counter (PC)
- CPU Registers (general-purpose, SP, flags)
- Memory management info
- Accounting info

Key Points

- **Pure overhead** → No useful work done during context switch.
- Triggered by: interrupt, system call, preemption, process termination.
- Switch time depends on number of registers and hardware support.

```
P0 Running
  ↓ interrupt
Save P0 state → PCB0
Load P1 state ← PCB1
P1 Running
  ↓ interrupt
Save P1 state → PCB1
Load P0 state ← PCB0
P0 Resumes
```

In One Line

Context switching saves the old process state and restores the new process state — essential but costly overhead.

25. Scheduling Criteria

Criterion	Goal	Description
CPU Utilization	Maximize	Keep CPU as busy as possible (40–90%)
Throughput	Maximize	Processes completed per unit time
Turnaround Time (TAT)	Minimize	Submission to completion time
Waiting Time (WT)	Minimize	Time spent in Ready queue
Response Time (RT)	Minimize	Time from request to first response

Formulas

```
TAT = Completion Time (CT) - Arrival Time (AT)
WT  = TAT - Burst Time (BT)
RT  = First CPU time - Arrival Time
Avg WT = Sum of all WT / Number of processes
Avg TAT = Sum of all TAT / Number of processes
```

In One Line

Good scheduling maximizes CPU utilization and throughput while minimizing TAT, WT, and RT.

26. FCFS Scheduling

Definition

First Come First Served — the process that **arrives first gets the CPU first**.

Non-preemptive.

Characteristics

- **Simple** and easy to implement.
- **Convoy effect** → Short jobs wait behind long ones → high average WT.
- **No starvation**.

Example

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	2	10	8	6

Gantt: | P1(0-5) | P2(5-8) | P3(8-10) |

Avg WT = $(0+4+6)/3 = 3.33$ | Avg TAT = $(5+7+8)/3 = 6.67$

In One Line

FCFS is simple but causes the convoy effect — long jobs delay short ones.

27. SJF Scheduling

Definition

Shortest Job First — selects the process with the **shortest burst time** from the Ready queue. Non-preemptive.

Characteristics

- **Optimal** average waiting time for non-preemptive scheduling.
- **Problem:** Requires knowing burst time in advance (estimated using exponential averaging).
- **Starvation** → Long processes may wait indefinitely.
- **Solution to starvation:** Aging.

In One Line

SJF gives optimal average WT but requires advance knowledge of burst time and can starve long processes.

28. SRTF Scheduling

Definition

Shortest Remaining Time First — preemptive version of SJF. If a new process arrives with a **shorter remaining burst time** than the current process, the CPU is preempted.

Characteristics

- **Optimal** average waiting time overall.
- **Most preemptive** → High context switch overhead.

- Still suffers from **starvation** for long processes.

In One Line

SRTF is preemptive SJF — optimal but with high context switch overhead and starvation risk.

29. Priority Scheduling

Definition

Each process has a **priority number**; CPU is given to the **highest-priority** process (lower number = higher priority in most systems). Can be preemptive or non-preemptive.

Key Points

- **Starvation** → Low-priority processes may never execute.
- **Aging** → Gradually increase priority of waiting processes to prevent starvation.
- **Preemptive**: If a higher-priority process arrives, current process is preempted.

In One Line

Priority scheduling assigns CPU based on priority; aging prevents starvation of low-priority processes.

30. Round Robin Scheduling

Definition

Round Robin (RR) — each process gets the CPU for a fixed **time quantum (q)**. After the quantum, it is preempted and moved to the end of the Ready queue. Preemptive.

Characteristics

- **Fair** → No starvation; every process gets CPU time.
- **Performance depends on quantum size:**
- Large q → behaves like FCFS.

- Small $q \rightarrow$ high context switch overhead.
- Rule of thumb: **80% of CPU bursts should be shorter than q .**
- Best for **time-sharing / interactive systems.**

In One Line

Round Robin gives each process a time slice (quantum) in rotation — fair, but quantum size affects performance.

SECTION 4 — THREADS & IPC

31. Thread Concept

Definition

A **thread** is the **smallest unit of CPU execution** within a process. A process can have multiple threads, each with its own PC, registers, and stack, but all sharing code, data, and files.

Process

```
■■■ Thread 1 (own: PC, stack, registers)
■■■ Thread 2 (own: PC, stack, registers)
■■■ Thread 3 (own: PC, stack, registers)
    ■■■ All share: code, data, heap, files
```

In One Line

A thread is a lightweight unit of execution within a process, sharing the process's resources but with its own control flow.

32. Process vs Thread

Feature	Process	Thread
Definition	Program in execution	Segment of a process
Address Space	Separate	Shared within process
Creation Overhead	High	Low
Communication	IPC required	Shared memory directly
Context Switch Cost	Expensive	Cheaper
Independence	Fully independent	Depends on process
Failure Impact	Isolated	Can crash entire process

In One Line

Threads are lighter than processes — they share memory and are cheaper to create and switch.

33. Benefits of Multithreading

- **Responsiveness** → UI thread stays responsive while background threads work.
- **Resource Sharing** → Threads share memory; no extra IPC needed.
- **Economy** → Cheaper to create and context-switch than processes.
- **Scalability** → Threads can run in parallel on multiple CPU cores.

In One Line

Multithreading improves responsiveness, resource sharing, and scalability with lower overhead than multiprocessing.

34. User-Level vs Kernel-Level Threads

Feature	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Managed by	User-space thread library	OS Kernel
Kernel awareness	Kernel sees only one process	Kernel knows each thread
Speed	Faster (no kernel call)	Slower (kernel involvement)
Blocking	One blocked → all blocked	One blocked → others continue
Parallelism	No true parallelism	True parallelism on multicores
Example	Early POSIX pthreads	Linux tasks, Windows threads

In One Line

ULTs are fast but block all threads on I/O; KLTs allow true parallelism with kernel support.

35. Many-to-One Model

Definition

Multiple user threads map to one kernel thread.

[ULT1][ULT2][ULT3]



[Kernel Thread]

Key Points

- Thread management done in user space → **fast**.
- **One thread blocked** → **all threads blocked**.
- **No parallelism** on multicore CPUs.
- Rarely used today.

In One Line

Many-to-One is fast but blocking one ULT blocks all, and no multicore parallelism.

36. One-to-One Model

Definition

Each user thread maps to **one kernel thread**.

```
[ULT1] → [KT1]
[ULT2] → [KT2]
[ULT3] → [KT3]
```

Key Points

- **True parallelism** on multicores.
- One blocked thread doesn't affect others.
- Creating many user threads creates many kernel threads → **overhead**.
- Used by: **Linux, Windows**.

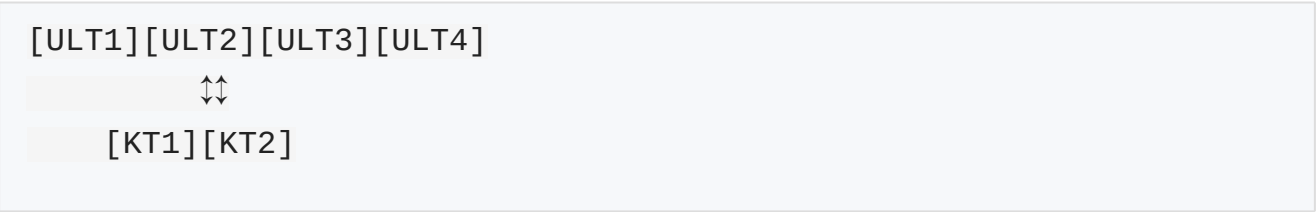
In One Line

One-to-One allows true parallelism; used by Linux and Windows, but has kernel thread overhead.

37. Many-to-Many Model

Definition

Multiple user threads map to multiple kernel threads ($\leq$ ULTs).



Key Points

- OS creates **enough kernel threads** as needed.
- Allows **true parallelism** without excessive kernel thread creation.
- **Most flexible** but complex to implement.

In One Line

Many-to-Many is the most flexible model — balances parallelism and overhead.

38. Inter-Process Communication (IPC)

Definition

IPC is a set of mechanisms that allow processes to **communicate and synchronize** with each other.

Two Main Models

Model	Description	Speed	Complexity
Shared Memory	Processes share a region of memory	Fastest	Need synchronization

Message Passing	Processes exchange messages via kernel	Slower	Easier (no sync needed)
------------------------	--	--------	-------------------------

IPC Mechanisms

Mechanism	Description
Shared Memory	Region of memory shared; fastest IPC
Message Queue	Messages stored in a queue by kernel
Pipes	Unidirectional byte stream between related processes
Named Pipes (FIFO)	Pipe accessible by name; unrelated processes
Sockets	Network/local communication endpoint
Signals	Asynchronous notifications
Semaphores	Synchronization primitives

In One Line

IPC allows processes to communicate via shared memory (fast) or message passing (safe); pipes, sockets, and signals are common mechanisms.

39. Shared Memory

Definition

Shared memory is a region of memory that is **mapped into the address spaces of multiple processes**, allowing them to read and write directly.

Key Points

- **Fastest IPC** — no kernel involvement after initial setup.
- Requires **synchronization** (semaphores/mutexes) to prevent race conditions.
- Producer-Consumer problem is a classic shared memory example.
- UNIX: `shmget()`, `shmat()`, `shmdt()`, `shmctl()`.

In One Line

Shared memory is the fastest IPC; processes share a memory region but must synchronize access.

40. Message Passing

Definition

Message passing allows processes to communicate by **sending and receiving messages** through the OS kernel, without sharing memory.

Key Points

- **Easier** than shared memory — no synchronization needed by programmer.
- **Slower** — kernel involved in each message (data copied twice: sender→kernel→receiver).
- Useful in **distributed systems** where shared memory isn't possible.
- Types:
 - **Direct** → `send(P, msg) / receive(Q, msg)` — must know each other's identity.
 - **Indirect (Mailboxes)** → Messages sent to a mailbox/port; receiver picks from mailbox.
 - **Synchronous (Blocking)** → Sender blocks until receiver receives.
 - **Asynchronous (Non-blocking)** → Sender continues without waiting.

In One Line

Message passing lets processes communicate safely via kernel without shared memory — simpler but slower.

SECTION 5 — SYNCHRONIZATION

41. Process Synchronization

Definition

Process synchronization is the coordination of processes that **share data** to ensure **data consistency** and correct ordering of execution.

Why Needed

- Concurrent processes accessing shared data can lead to **inconsistency**.
- Without sync: final value depends on execution order → **unpredictable**.

In One Line

Process synchronization ensures shared data is accessed in a controlled manner to maintain consistency.

42. Race Condition

Definition

A **race condition** occurs when **multiple processes access and manipulate shared data concurrently**, and the outcome depends on the **order of execution**.

Example

Two processes increment a shared counter (count = 5):

P1 reads count = 5

P2 reads count = 5

```
P1 increments: count = 6, writes 6
P2 increments: count = 6, writes 6
Result: count = 6 instead of 7 ← WRONG!
```

In One Line

A race condition produces incorrect results because multiple processes access shared data without synchronization.

43. Critical Section

Definition

A **critical section** is a **code segment** where a process accesses **shared resources** (variables, files, hardware). Only one process should execute in its critical section at a time.

Structure

```
Entry Section      // Request permission to enter
Critical Section   // Access shared resource
Exit Section       // Release permission
Remainder Section  // Non-critical code
```

In One Line

The critical section is the shared-resource access code that must be executed by only one process at a time.

44. Critical-Section Requirements

Any valid solution to the critical-section problem must satisfy **all three**:

Requirement	Description
Mutual Exclusion	Only one process in CS at a time

Progress	If no process is in CS, the next process to enter must be decided without indefinite postponement
Bounded Waiting	A limit must exist on how many times others can enter CS before a waiting process gets its turn

In One Line

A valid CS solution must ensure mutual exclusion, progress, and bounded waiting.

45. Peterson's Solution

Definition

Peterson's solution is a **software-based** mutual exclusion solution for **two processes** using two shared variables.

Variables

- `flag[2]` → Indicates if a process wants to enter CS (true/false).
- `turn` → Whose turn it is (0 or 1).

Code for Process P_i (j = other process)

```
flag[i] = true;           // I want to enter
turn = j;                 // But let j go first
while (flag[j] && turn == j); // Wait
// CRITICAL SECTION
flag[i] = false;          // Done
// REMAINDER SECTION
```

Key Points

- Satisfies all three requirements for **2 processes**.
- **Not guaranteed** on modern architectures (instruction reordering).
- Works correctly assuming **sequential execution**.

In One Line

Peterson's solution uses `flag[]` and `turn` to achieve mutual exclusion for two processes in software.

46. Atomic Operations

Definition

An **atomic operation** is one that **executes completely as a single, indivisible unit** — no other process can observe or interrupt it midway.

Key Points

- Used to implement **lock-free synchronization**.
- Provided by hardware instructions.
- **No partial state visible** to other processes.
- Examples: Test-and-Set, Compare-and-Swap, Fetch-and-Add.

In One Line

Atomic operations execute as a **single, uninterruptible unit** — the foundation of hardware synchronization.

47. Test-and-Set (TAS)

Definition

Test-and-Set is a hardware atomic instruction that **reads a variable, sets it to true, and returns the old value** — all atomically.

```
boolean test_and_set(boolean *target) {  
    boolean rv = *target;  
    *target = true;  
    return rv;  
}
```

Usage (Mutex Lock)

```
// lock = false initially (unlocked)  
while (test_and_set(&lock)); // Spin until lock acquired  
// CRITICAL SECTION  
lock = false; // Release lock
```

Key Points

- Satisfies **mutual exclusion**.
- Does **not** guarantee **bounded waiting** by itself.
- Causes **busy-waiting (spinlock)** → CPU wasted while waiting.

In One Line

TAS atomically reads and sets a boolean — used to implement spinlocks.

48. Compare-and-Swap (CAS)

Definition

Compare-and-Swap is a hardware atomic instruction that compares a memory value to an expected value; if equal, swaps it with a new value.

```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
    if (*value == expected)  
        *value = new_value;  
    return temp;  
}
```

```
}
```

Key Points

- More powerful than TAS; supports **lock-free** data structures.
- Used in modern CPUs (x86: `CMPXCHG`, ARM: `LDREX/STREX`).
- Foundation of **non-blocking algorithms**.

In One Line

CAS atomically checks and updates a value — used in lock-free algorithms and modern synchronization.

49. Semaphores

Definition

A **semaphore** is an integer synchronization variable accessed only through two **atomic operations**: `wait()` and `signal()`.

Operations

```
wait(S):          signal(S):
  while (S <= 0);   S++;
  S--;
```

Blocking Semaphore (No Busy-Wait)

```
wait(S):          signal(S):
  S.value--;       S.value++;
  if (S.value < 0) {   if (S.value <= 0) {
    add to waitlist    wakeup(P from list)
    block()            }
  }
```

Usage Patterns

- **Mutual exclusion:** Init `S=1`; `wait()` before CS, `signal()` after.
- **Synchronization:** Init `S=0`; P2 does `wait(S)` → P1 signals with `signal(S)` after completing task.

In One Line

Semaphores are atomic integer variables controlling access using `wait()` and `signal()` — used for both mutual exclusion and synchronization.

50. Binary vs Counting Semaphore

Feature	Binary Semaphore (Mutex)	Counting Semaphore
Value Range	0 or 1	0 to N
Purpose	Mutual exclusion (1 resource)	Managing N resources
Init Value	1 (unlocked)	N (number of resources)
Alternative Name	Mutex lock	General semaphore

In One Line

Binary semaphore (0/1) gives mutual exclusion; counting semaphore (0–N) manages N resource instances.

SECTION 6 — DEADLOCK

51. Deadlock

Definition

A **deadlock** is a situation where **a set of processes are permanently blocked**, each waiting for a resource held by another process in the set — none can proceed.

Simple Example

```
P1 holds R1, waits for R2
P2 holds R2, waits for R1
→ Neither can proceed → DEADLOCK
```

In One Line

Deadlock is a circular waiting situation where no process can proceed.

52. Four Necessary Conditions of Deadlock

All four must hold **simultaneously** for deadlock to occur:

Condition	Description
Mutual Exclusion	At least one resource is non-shareable; only one process at a time can use it
Hold and Wait	A process holds at least one resource while waiting for additional resources held by others
No Preemption	Resources cannot be forcibly taken away; only released voluntarily
Circular Wait	A circular chain: P1 waits for P2, P2 waits for P3, ..., Pn waits for P1

In One Line

Deadlock requires all four: mutual exclusion, hold-and-wait, no preemption, and circular wait.

53. Deadlock Prevention

Definition

Deadlock prevention eliminates deadlock by ensuring **at least one of the four necessary conditions cannot hold**.

Condition to Eliminate	Strategy
Mutual Exclusion	Make resources shareable (not always possible)
Hold and Wait	Request all resources at once before starting; or release all before requesting new ones
No Preemption	If a process waiting for a resource cannot get it, release all its current resources
Circular Wait	Impose a total ordering on resource types; always request in increasing order

Drawbacks

- Low resource utilization.
- Possible starvation.

In One Line

Deadlock prevention eliminates at least one necessary condition — but reduces efficiency.

54. Deadlock Avoidance

Definition

Deadlock avoidance grants resources only when the resulting state is **safe** — a safe state guarantees the system can complete all processes in some order.

Safe State

- **Safe State** → There exists a **safe sequence** in which every process can finish using available + currently held resources.
- **Unsafe State** → No safe sequence exists; deadlock may occur.

Algorithm Used

- **Banker's Algorithm** for multiple resource instances.
- **Resource Allocation Graph** algorithm for single-instance resources.

In One Line

Deadlock avoidance uses advance resource need info to stay in a safe state; uses Banker's Algorithm.

55. Deadlock Detection

Definition

Deadlock detection allows deadlocks to occur; periodically runs an algorithm to **detect if a deadlock exists** in the system.

Detection Methods

- **Single instance resources** → Detect cycle in **Resource Allocation Graph (RAG)**.
- **Multiple instance resources** → Use **detection algorithm** (similar to Banker's safety check).

When to Run

- After every resource request (expensive but immediate detection).

- Periodically (e.g., every N minutes or when CPU utilization drops).

In One Line

Deadlock detection allows deadlocks to form, then identifies them using RAG cycle detection or a detection algorithm.

56. Deadlock Recovery

Definition

After a deadlock is detected, the OS must **recover** by breaking the circular wait.

Recovery Methods

1. Process Termination

- **Abort all** deadlocked processes → drastic, loses progress.
- **Abort one at a time** → re-run detection after each; less drastic.
- Victim selection: lowest priority, least work done, most resources held.

2. Resource Preemption

- **Select a victim** → take its resources; give to blocked processes.
- **Rollback** → return victim to a safe state (checkpoint-restart).
- **Starvation prevention** → don't always pick the same victim; include rollback count in cost.

In One Line

Deadlock recovery either terminates processes or preempts resources from a victim to break the circular wait.

57. Resource Allocation Graph (RAG)

Definition

A **RAG** is a directed graph used to detect deadlock in systems with **single-instance resources**.

Components

- **Circle (●)** → Process node.
- **Rectangle (■)** → Resource node.
- **Request edge (P → R)** → Process P is requesting resource R.
- **Assignment edge (R → P)** → Resource R is assigned to process P.

Deadlock Detection Rule

- **Cycle exists** → **Deadlock** (for single-instance resources).
- **No cycle** → **No deadlock**.

P1 → R1 → P2 → R2 → P1 ← Cycle = Deadlock

In One Line

RAG uses directed edges to represent resource requests and assignments; a cycle means deadlock (single-instance).

58. Safe State vs Unsafe State

State	Description	Deadlock?
Safe State	A safe sequence exists; all processes can finish	No
Unsafe State	No safe sequence; deadlock may occur	Possible
Deadlock	Circular wait; no process can proceed	Yes

Safe State → Unsafe State → Deadlock

↑

(Not all unsafe states are deadlocks)

In One Line

Safe state guarantees all processes can finish; unsafe state may lead to deadlock; deadlock is circular wait.

59. Banker's Algorithm

Definition

Banker's Algorithm is a deadlock avoidance algorithm for systems with **multiple resource instances**. It grants a resource request only if the system remains in a **safe state**.

Data Structures (n processes, m resources)

Structure	Description
Available[m]	Available instances of each resource type
Max[n][m]	Maximum demand of each process
Allocation[n][m]	Currently allocated to each process
Need[n][m]	Remaining need = Max – Allocation

Safety Algorithm

1. Work = Available; Finish[i] = false for all i
2. Find P_i : Finish[i]=false AND Need[i] ≤ Work
3. Work = Work + Allocation[i]; Finish[i] = true
4. Repeat step 2-3 until no such P_i
5. If all Finish[i] = true → SAFE; else UNSAFE

Resource-Request Algorithm

```
Request[i] ≤ Need[i]?      → else ERROR
Request[i] ≤ Available?    → else WAIT
Pretend to allocate → run Safety Algorithm
If SAFE → grant; If UNSAFE → rollback and WAIT
```

In One Line

Banker's Algorithm grants requests only when a safe sequence exists — simulates allocation and checks safety.

60. Safe Sequence & Deadlock Numericals

Template

Given: Available, Max, Allocation → **Compute** Need = Max – Allocation.

Process	Allocation (A B C)	Max (A B C)	Need (A B C)
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Available = (3, 3, 2)

Step	Process	Work Before	Work After
------	---------	-------------	------------

1	P1	3,3,2	5,3,2
2	P3	5,3,2	7,4,3
3	P4	7,4,3	7,4,5
4	P0	7,4,5	7,5,5
5	P2	7,5,5	10,5,7

Safe Sequence: P1 → P3 → P4 → P0 → P2 ✓

In One Line

Find safe sequence by repeatedly allocating Work to any process whose Need $\leq$ Work until all finish.

SECTION 7 — MEMORY MANAGEMENT

61. Memory Management

Definition

Memory management is the OS function that handles the **allocation, tracking, and reclamation** of main memory (RAM) for processes.

Goals

- **Maximize memory utilization.**
- **Allow multiple processes** in memory simultaneously.
- **Protect** one process's memory from another.
- **Support virtual memory** for programs larger than physical RAM.

In One Line

Memory management allocates, protects, and reclaims RAM to enable efficient multiprogramming.

62. Logical vs Physical Address

Feature	Logical Address	Physical Address
Also called	Virtual address	Real address
Generated by	CPU (during execution)	MMU (hardware translation)
Seen by	User program	Memory hardware
Binding	Runtime (execution time)	After translation

In One Line

Logical address is what the CPU generates; physical address is the actual RAM location — MMU translates between them.

63. Address Binding

Definition

Address binding is the mapping of a program's **symbolic addresses** → **logical addresses** → **physical addresses**.

Stages

Stage	When	Description
-------	------	-------------

Compile Time	During compilation	Absolute addresses hardcoded; must know final location
Load Time	When loaded into memory	Relocatable code; physical address fixed at load
Execution Time	During runtime	Address can change during execution; needs MMU

In One Line

Address binding converts program addresses to physical memory addresses — at compile, load, or execution time.

64. Memory Management Unit (MMU)

Definition

The **MMU** is a hardware device that **translates logical addresses to physical addresses** at runtime.

Basic Translation (Relocation Register)

$$\text{Physical Address} = \text{Logical Address} + \text{Relocation Register (Base)}$$

Example

Base = 14000, Logical Address = 346
 Physical Address = 14000 + 346 = 14346

Key Points

- Also enforces memory protection using a **limit register**.

- If Logical Address $\geq$ Limit $\rightarrow$ **Segmentation Fault (protection error)**.
- Part of the CPU chip in modern systems.

In One Line

MMU hardware translates logical to physical addresses at runtime using a base + limit register.

65. Contiguous Memory Allocation

Definition

Each process is allocated a **single contiguous block** of memory.

Types

- **Single Partition** $\rightarrow$ OS in one part, user in the rest.
- **Fixed Partitioning** $\rightarrow$ Memory divided into fixed-size partitions; one process per partition $\rightarrow$ **internal fragmentation**.
- **Variable Partitioning** $\rightarrow$ Partitions created dynamically to fit each process $\rightarrow$ **external fragmentation**.

In One Line

Contiguous allocation gives each process a single block; fixed partitioning wastes with internal fragmentation; variable causes external fragmentation.

66. First Fit, Best Fit & Worst Fit

Technique	Strategy	Pros	Cons
First Fit	Allocate first hole big enough	Fast	May fragment large holes

Best Fit	Allocate smallest sufficient hole	Minimizes waste	Leaves tiny leftover holes; slow
Worst Fit	Allocate largest hole	Leaves large leftovers	Slow; wastes large holes
Next Fit	Like First Fit but starts from last point	Distributes load	Similar issues to First Fit

First Fit and Best Fit are generally better than Worst Fit in practice.

In One Line

First Fit is fastest; Best Fit minimizes waste per allocation; Worst Fit is generally the poorest choice.

67. Internal vs External Fragmentation

Feature	Internal Fragmentation	External Fragmentation
Definition	Wasted space inside an allocated partition	Enough total free memory exists but scattered in small non-contiguous holes
Caused by	Fixed partitioning, paging	Variable partitioning, segmentation
Solution	Smaller block sizes, paging	Compaction, paging

In One Line

Internal fragmentation wastes space inside partitions; external fragmentation scatters free space into unusable holes.

68. Compaction

Definition

Compaction is the process of **shuffling all processes to one end** of memory and **combining all free holes** into a single large block.

Key Points

- **Eliminates external fragmentation.**
- Very **expensive** → requires relocating all processes.
- Only possible if **relocation is dynamic** (execution-time binding).
- Reduces performance during compaction.

In One Line

Compaction moves all processes to one end to merge scattered free holes — expensive but cures external fragmentation.

69. Swapping

Definition

Swapping is temporarily **moving a process from main memory to a backing store (swap space on disk)** and later bringing it back.

Key Points

- Allows **more processes** to run than physical memory alone allows.
- **Swap-out:** Move process to disk. **Swap-in:** Bring process back to memory.
- Modern OS uses **lazy swapping** (swap only when memory is low).
- Major bottleneck: **disk I/O speed** is much slower than RAM.

```
Memory: [P1][P2][P3]
        ↓ (swap P2 out)
Disk:    P2 in swap space
Memory: [P1][ ][P3]
        ↑ (swap P4 in)
Memory: [P1][P4][P3]
```

In One Line

Swapping moves processes to disk to free memory, enabling more processes than physical memory supports.

70. Paging

Definition

Paging divides physical memory into fixed-size **frames** and logical memory into same-size **pages**. Pages of a process are loaded into any available frames.

Key Points

- **No external fragmentation** (frames allocated anywhere).
- Small **internal fragmentation** (last page may be partially full).
- OS maintains a **page table** per process (maps page # → frame #).

Address Translation

```
Logical Address: [Page Number p | Offset d]
                  ↓ Page Table[p] = Frame f
Physical Address: [Frame Number f | Offset d]
```

In One Line

Paging eliminates external fragmentation by mapping fixed-size logical pages to any physical frames.

SECTION 8 — VIRTUAL MEMORY

71. Pages, Frames & Page Table

Term	Description
Page	Fixed-size block of a process's logical address space
Frame	Fixed-size block of physical memory (same size as page)
Page Table	Per-process table mapping page number → frame number
Page Size	Typically 4KB; must be a power of 2

Key Points

- Logical address space is divided into pages; physical memory into frames.
- Page table translates logical to physical for each memory access.
- Large processes need large page tables → multilevel paging used.

In One Line

Pages are logical blocks, frames are physical blocks, and the page table maps one to the other.

72. Paging Address Translation

Translation Process

Logical Address (32-bit example, 4KB pages):

Bits 31-12 = Page Number (p) = 20 bits $\rightarrow 2^{20}$ pages

Bits 11-0 = Page Offset (d) = 12 bits $\rightarrow$ 4KB page size

Physical Address:

Frame Number (f) from Page Table[p]

Physical Address = $f \times 4096 + d$

Example

- Page size = 4KB = 4096 bytes.
- Logical address = 8300.
- Page number $p = 8300 / 4096 = 2$ (page 2).
- Offset $d = 8300 \% 4096 = 108$.
- Page Table[2] = Frame 5.
- Physical address = $5 \times 4096 + 108 = 20588$.

In One Line

Address translation splits logical address into page# and offset; page table gives frame#; combine for physical address.

73. TLB (Translation Lookaside Buffer)

Definition

TLB is a **small, fast hardware cache** inside the CPU that stores recent page table entries to speed up address translation.

TLB Operation

CPU generates logical address

↓

Check TLB for page number

■■■■ TLB Hit (found) $\rightarrow$ get frame directly $\rightarrow$ fast (1 memory access)

■■■ TLB Miss (not found) → access page table in RAM → slow (2 memory access)
→ update TLB with new entry

Key Points

- TLB hit ratio typically **90–99%**.
- **TLB Flush** needed on context switch (different process = different page table).
- ASID (Address Space Identifier) avoids full flush in some architectures.

In One Line

TLB is a hardware cache for page table entries; a hit gives fast translation, a miss requires a slow page table access.

74. Effective Access Time (EAT)

Definition

EAT is the **average time to access a memory location**, accounting for TLB hits and misses.

Formula

Let:

α = TLB hit ratio (e.g., 0.9)

t = TLB access time

m = Memory access time

$$\begin{aligned} \text{EAT} &= \alpha \times (t + m) + (1 - \alpha) \times (t + 2m) \\ &= t + (2 - \alpha) \times m \end{aligned}$$

Example

- TLB access = 20ns, Memory = 100ns, Hit ratio = 90%

$$\begin{aligned} \text{EAT} &= 0.9 \times (20 + 100) + 0.1 \times (20 + 100 + 100) \\ &= 0.9 \times 120 + 0.1 \times 220 \\ &= 108 + 22 = 130 \text{ ns} \end{aligned}$$

In One Line

EAT = weighted average of TLB hit access time and TLB miss access time.

75. Multilevel Paging

Definition

When the page table itself is too large to fit in memory, it is **divided into pages** — creating multiple levels of page tables.

Two-Level Paging (32-bit, 4KB pages)

```
Logical Address: [p1 (10 bits) | p2 (10 bits) | d (12 bits)]
                    ↓
                Outer Page Table
                    ↓
                Inner Page Table
                    ↓
                Physical Frame
```

Key Points

- Used when address space is **large but sparse** (most page table entries unused).
- Common on **64-bit systems**: 4-level paging (x86-64 uses PML4 → PDPT → PD → PT).
- Each level adds a memory access → more TLB misses.

In One Line

Multilevel paging breaks the page table into a hierarchy of tables to handle large sparse address spaces.

76. Segmentation

Definition

Segmentation divides a program into **logical segments** (code, data, stack, heap) of **variable size**, each with a base and a limit.

Address Translation

```
Logical Address: [Segment Number s | Offset d]
  ↓ Segment Table[s] → {Base, Limit}
  ↓ Check: if d < Limit → Physical = Base + d
                else → Segmentation Fault
```

Segmentation vs Paging

Feature	Paging	Segmentation
Division	Fixed-size pages	Variable-size segments
Fragmentation	Internal (small)	External
User view	Physical	Logical (matches program structure)
Hardware	Page table	Segment table

In One Line

Segmentation divides memory into logical variable-size segments; no internal fragmentation but external fragmentation occurs.

77. Virtual Memory

Definition

Virtual memory allows a program to run even if it is **not entirely in physical memory** — only the needed portions are loaded; the rest stays on disk.

Key Benefits

- Programs can be **larger than physical memory**.
- More processes can run simultaneously.
- **Faster process startup** (only load what's needed).

Implementation

- Uses **demand paging** (load pages on demand).
- OS maintains a **valid-invalid bit** in page table entries.
- **Valid** = page is in memory.
- **Invalid** = page is on disk (not loaded yet).

In One Line

Virtual memory lets programs exceed physical RAM size by keeping only needed pages in memory and the rest on disk.

78. Demand Paging

Definition

Demand paging loads a page into memory **only when it is accessed** (demanded) — not at process start.

Key Points

- Uses **lazy swapper (pager)** — only brings pages that are needed.
- Reduces initial load time and memory usage.
- **Valid-invalid bit** tracks which pages are in memory.
- If a process accesses a page not in memory → **page fault**.

In One Line

Demand paging loads pages only when accessed, reducing memory usage and startup time.

79. Page Fault & Page-Fault Handling

Definition

A **page fault** occurs when a process accesses a page that is **not currently in physical memory** (valid-invalid bit = invalid).

Page-Fault Handling Steps

1. Process accesses a page
2. Page table: invalid bit → Page Fault (trap to OS)
3. OS checks: is the reference valid? (legal address?)
4. If invalid reference → terminate process (segfault)
5. If valid: find a free frame in memory
6. Load the required page from disk into the frame
7. Update page table: set valid bit; record frame number
8. Restart the instruction that caused the fault

Key Points

- **Page fault rate (p)** → Effective Access Time = $(1-p) \times \text{memory time} + p \times \text{page fault time}$.
- High page fault rate → poor performance (thrashing if extreme).

In One Line

A page fault traps to the OS, loads the missing page from disk, updates the page table, and restarts the instruction.

80. Page Replacement Algorithms

Definition

When a page fault occurs and **no free frame is available**, the OS must **replace an existing page** in memory.

Algorithms

Algorithm	Strategy	Faults	Anomaly
FIFO	Replace oldest page	High	Bélády's anomaly
Optimal (OPT)	Replace page not used for longest future time	Minimum (benchmark)	None
LRU	Replace least recently used page	Near-optimal	None
Second Chance	FIFO with reference bit; give second chance	Better than FIFO	None
Clock	Circular second-chance using clock hand	Practical LRU approx	None

In One Line

Page replacement selects a victim page to evict; OPT is best theoretically; LRU is best practically.

SECTION 9 — PAGE REPLACEMENT & FILE SYSTEM

81. FIFO Page Replacement

Definition

First-In First-Out — replaces the **oldest page** (the one that has been in memory the longest).

Example

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 | Frames = 3

Ref	Frames	Find?
1	1 - -	F
2	1 2 -	F
3	1 2 3	F
4	4 2 3	F (replace 1)
1	4 1 3	F (replace 2)
2	4 1 2	F (replace 3)

5		5 F 1 (replace 2 4)
...		

Total faults = 9

In One Line

FIFO replaces the oldest page — simple but may evict heavily-used pages; suffers from Bélády's anomaly.

82. Optimal Page Replacement

Definition

Optimal (OPT) — replaces the page that will **not be used for the longest time in the future**.

Key Points

- **Lowest possible page faults** — theoretical benchmark.
- **Cannot be implemented** in practice (requires future knowledge).
- Used to **compare and evaluate** other algorithms.

In One Line

OPT has the minimum page faults but is unimplementable; used as a benchmark for other algorithms.

83. LRU Page Replacement

Definition

Least Recently Used — replaces the page that **has not been used for the longest time in the past**.

Implementation Methods

- **Counter-based** → Each page has a timestamp of last access; replace smallest timestamp.
- **Stack-based** → Most recently used on top; LRU always at bottom; no search needed.

Key Points

- Good approximation of Optimal.
- **No Bélády's anomaly.**
- Expensive to implement precisely (hardware support or software overhead).

In One Line

LRU replaces the least recently used page — a good approximation of Optimal with no Bélády's anomaly.

84. Second Chance / Clock Algorithm

Definition

Second Chance is a modified FIFO algorithm that gives a page a **second chance** before replacing it, using a **reference bit**.

How It Works

1. Pages arranged in a **circular queue (clock)**.
2. A **clock hand** scans pages.
3. If reference bit = **0** → **replace** this page.
4. If reference bit = **1** → set to 0 (give second chance), advance hand.
5. Repeat until a page with reference bit = 0 is found.

In One Line

Second Chance (Clock) modifies FIFO with a reference bit — gives pages a second chance before eviction.

85. Bélády's Anomaly

Definition

Bélády's Anomaly is the phenomenon where **increasing the number of frames causes more page faults** (counterintuitive behavior).

Key Points

- Occurs only in **FIFO** page replacement.
- **Does NOT occur** in LRU, OPT (they have stack property).
- Stack property: set of pages in memory with n frames $\subseteq$ set with $n+1$ frames.

Example

FIFO with reference string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5: - 3 frames: 9 page faults - 4 frames: 10 page faults ← **More faults with more frames!**

In One Line

Bélády's Anomaly: FIFO can get more page faults with more frames — increasing frames doesn't always help.

86. Thrashing

Definition

Thrashing occurs when a process spends **more time paging (page faults) than executing** — CPU utilization drops drastically.

Cause

```
Too many processes in memory
↓ Each gets too few frames
↓ High page fault rate
↓ CPU sees low utilization
↓ OS adds more processes (wrong decision)
↓ Even more thrashing
```

Control

- **Working-Set Model** → Give each process frames $\geq$ its working set size.
- **Page-Fault Frequency (PFF)** → Monitor fault rate; add/remove frames.
- **Reduce degree of multiprogramming** → Swap out some processes.

In One Line

Thrashing = spending more time paging than executing; caused by too few frames; controlled by working-set model.

87. Working-Set Model

Definition

The **working set** of a process is the set of pages it is **actively using** within a recent time window Δ (working-set window).

Key Points

- **WS(t, Δ)** = set of pages referenced in the last Δ time units.
- OS allocates frames $\geq |WS|$ to each process.
- **Total demand D** = sum of all working sets.
- If $D >$ total frames → **thrash** → swap out a process.
- As $\Delta \rightarrow \infty$: WS = all pages. As $\Delta \rightarrow 0$: WS = current page only.

In One Line

Working-set model allocates frames based on a process's active page set, preventing thrashing by matching supply to demand.

88. File Concept & File Operations

File Definition

A **file** is a named collection of related information stored on secondary storage — the smallest logical unit of storage.

File Attributes

- **Name, Identifier, Type, Location, Size, Permissions, Timestamps.**

File Operations

Operation	Description
Create	Allocate space; add directory entry
Open	Make file accessible; return file descriptor
Read	Read data at current read pointer
Write	Write data at current write pointer
Seek	Reposition the file pointer
Delete	Remove directory entry; free space
Truncate	Clear content; keep attributes; set size = 0
Close	Release file descriptor; flush buffers

In One Line

A file is a named unit of storage; file operations include create, open, read, write, seek, delete, and close.

89. File Access Methods

Method	Description	Use Case
Sequential	Read/write bytes in order from start	Text files, tape storage
Direct (Random)	Access any block directly by position	Databases, disk storage
Indexed	Use an index block to find specific records	Large databases with key-based lookup

In One Line

Sequential access is ordered; direct access jumps to any position; indexed access uses an index for fast key-based lookup.

90. File Allocation Methods

Method	Description	Pros	Cons
Contiguous	File occupies consecutive blocks	Fast sequential & random access	External fragmentation; need to know size upfront
Linked	Each block has pointer to next block	No external fragmentation; file can grow	Poor random access; pointer overhead
Indexed	Index block contains all data block pointers	Fast random access; no external fragmentation	Index block overhead; small file wastes index

In One Line

Contiguous is fast but inflexible; linked is flexible but slow; indexed is fast and flexible with overhead.

SECTION 10 — I/O, DISK & MODERN OS

91. Contiguous, Linked & Indexed File Allocation

(Detailed comparison — see Topic 90 above for table.)

Contiguous

```
File A: [Block 0][Block 1][Block 2]
File B: [Block 5][Block 6][Block 7][Block 8]
```

Linked

```
Block 2 → Block 7 → Block 14 → NULL
```

Indexed

```
Index Block: [3][7][12][25] → data at blocks 3, 7, 12, 25
```

In One Line

Contiguous: fast, fragmented. Linked: flexible, slow random access. Indexed: balanced, with index overhead.

92. File Control Block (FCB) / Inode

Definition

A **File Control Block (FCB)** (called **inode** in UNIX/Linux) is a **per-file data structure** stored on disk that contains all metadata about a file.

FCB / Inode Contents

Field	Description
File permissions	rwx for owner, group, others
File size	Current size in bytes
Timestamps	Created, accessed, modified
Owner UID/GID	User and group IDs
Data block pointers	Direct, indirect, double indirect pointers
Link count	Number of hard links

Key Points

- In UNIX, a **directory entry** maps a filename to an **inode number**.
- Inode does **not** store the filename.
- `ls -li` shows inode numbers.

In One Line

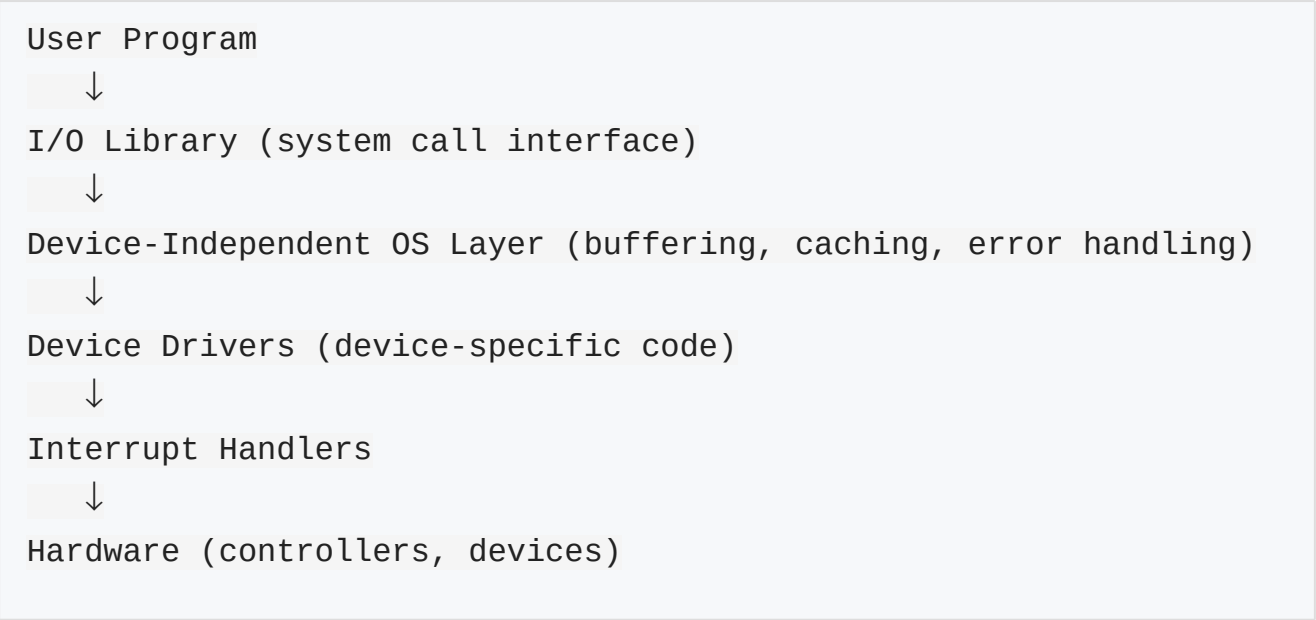
FCB/Inode stores all file metadata (permissions, size, block locations) — the OS's internal file descriptor.

93. I/O Management

Definition

I/O management is the OS function that **controls and coordinates all I/O devices** to provide uniform, efficient, and protected access.

I/O Software Layers



I/O Methods

Method	Description
Programmed I/O	CPU polls device status in a loop (busy waiting)
Interrupt-driven I/O	CPU notified via interrupt when I/O done
DMA	Device transfers data directly to/from memory; CPU interrupted only on completion

In One Line

I/O management abstracts hardware devices and provides efficient, uniform access using drivers, interrupts, and DMA.

94. Device Drivers & Device Controllers

Device Controller

- Hardware component that **operates a device** (disk controller, USB controller).
- Has **registers**: data-in, data-out, status, control.
- CPU communicates by reading/writing controller registers.

Device Driver

- OS **software module** that provides a **standard interface** to a specific device.
- Translates generic OS read/write calls into device-specific commands.
- Hides hardware complexity; one driver per device type.
- Runs in **kernel mode**.

Relationship

OS Kernel → Device Driver (software) → Device Controller (hardware) → Device

In One Line

Device driver = software interface to hardware; device controller = hardware that operates the device.

95. DMA (Direct Memory Access)

Definition

DMA is a hardware mechanism that allows I/O devices to **transfer data directly to/from main memory** without involving the CPU for each byte.

How DMA Works

1. CPU sets up DMA: source address, destination address, byte count
2. DMA controller takes over the bus

3. DMA transfers data directly between device and memory
4. CPU is FREE to do other work during transfer
5. DMA sends interrupt to CPU when transfer is complete
6. CPU handles completion interrupt

Key Points

- Dramatically improves **efficiency** for large data transfers (disk, network).
- **Bus mastering** → DMA controller takes control of the system bus.
- **Cycle stealing** → DMA steals bus cycles from CPU (slight slowdown).
- Compared to **programmed I/O**: CPU is freed from byte-by-byte data movement.

In One Line

DMA lets I/O devices transfer data directly to/from memory without CPU involvement per byte — interrupts only on completion.

96. Buffering vs Caching vs Spooling

Feature	Buffering	Caching	Spooling
Purpose	Temporary storage to match speed differences	Store frequently used data for fast access	Queue jobs for a slow, non-sharable device
Location	RAM (kernel buffer)	RAM (fast cache)	Disk (spool area)
Data	In-transit data	Copy of existing data	Pending output jobs
Example	Network packet buffer	File cache, TLB	Print spooler

Direction	Input or output	Read speedup	Output only
-----------	-----------------	--------------	-------------

In One Line

Buffering matches speeds; caching copies data for speed; spooling queues jobs for exclusive devices like printers.

97. Disk Scheduling Algorithms

Disk Access Time

Disk Access Time = Seek Time + Rotational Latency + Transfer Time

- **Seek Time** → Time for disk arm to move to correct track (dominant cost).
- **Rotational Latency** → Time for correct sector to rotate under head.
- **Transfer Time** → Time to actually read/write data.

Goal: Minimize seek time by optimizing order of disk request servicing.

Disk Queue: 98, 183, 37, 122, 14, 124, 65, 67 | Head at **53** | Disk: 0–199

Algorithm	Strategy	Total Movement
FCFS	Service in arrival order	640
SSTF	Nearest request first	236
SCAN	Move one direction; reverse at end	208
C-SCAN	One direction; jump to start on return	382

LOOK	Like SCAN; stop at last request	208
C-LOOK	Like C-SCAN; stop at last request	322

In One Line

Disk scheduling minimizes seek time; C-LOOK is the best practical algorithm for most workloads.

98. FCFS, SSTF, SCAN, C-SCAN, LOOK & C-LOOK

FCFS (First Come First Served)

- Serve in arrival order. **Fair** but high total movement. **No starvation.**

SSTF (Shortest Seek Time First)

- Serve nearest request first. **Low movement** but **starvation** for distant requests.

SCAN (Elevator Algorithm)

- Move in one direction; serve all requests; **reverse at end** (or disk boundary).
- **No starvation.** Uneven wait (recently-reversed area waits longest).

← 14, 37 | 53 → 65, 67, 98, 122, 124, 183, 199 → reverse ←

C-SCAN (Circular SCAN)

- Move in one direction; **jump to start** on reaching end (no service on return).
- More **uniform wait times** than SCAN.

LOOK

- Like SCAN but **stops at last request** in each direction (doesn't go to disk end).

- More efficient than SCAN.

C-LOOK (Circular LOOK)

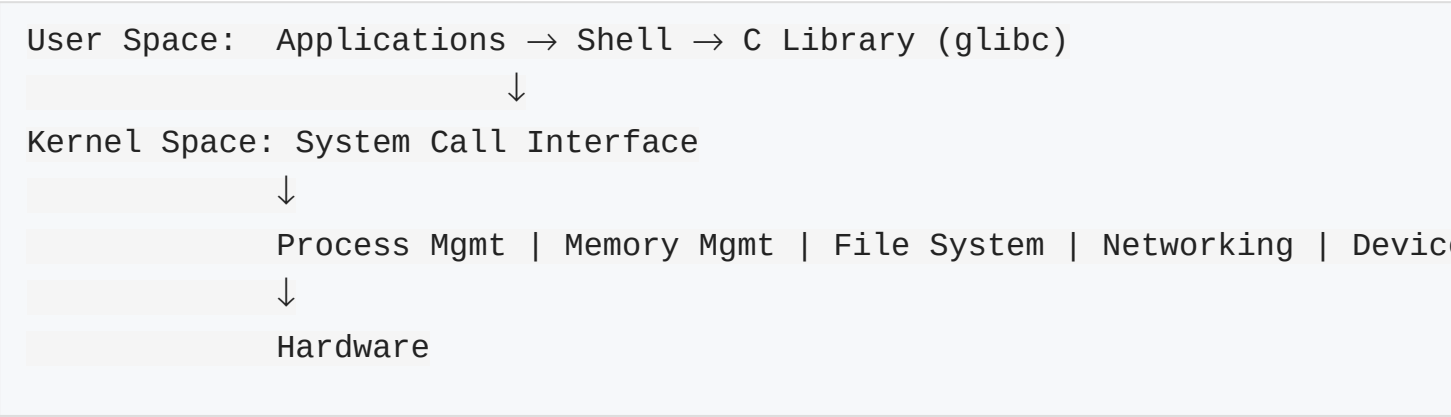
- Like C-SCAN but stops at **last request** each way.
- **Best practical algorithm** — efficient and uniform.

In One Line

FCFS=fair, SSTF=greedy, SCAN=elevator, C-SCAN=uniform, LOOK=efficient, C-LOOK=best practical.

99. Linux Operating System Basics

Architecture



Key Features

Feature	Details
Kernel Type	Monolithic with loadable modules
Scheduler	CFS (Completely Fair Scheduler)
Memory	Paging, buddy system, slab allocator

File System	ext4 (default), Btrfs, XFS; VFS abstraction
IPC	Pipes, signals, shared memory, sockets, semaphores
Security	DAC (UNIX permissions) + SELinux/AppArmor MAC
Process	task_struct (PCB), clone() for threads

Key Linux Commands (OS level)

Command	Purpose
<code>ps / top</code>	List processes / real-time process viewer
<code>kill</code>	Send signal to process
<code>lsmod / insmod / rmmod</code>	Manage kernel modules
<code>free / vmstat</code>	Memory usage
<code>df / mount</code>	File system usage / mounting
<code>strace</code>	Trace system calls

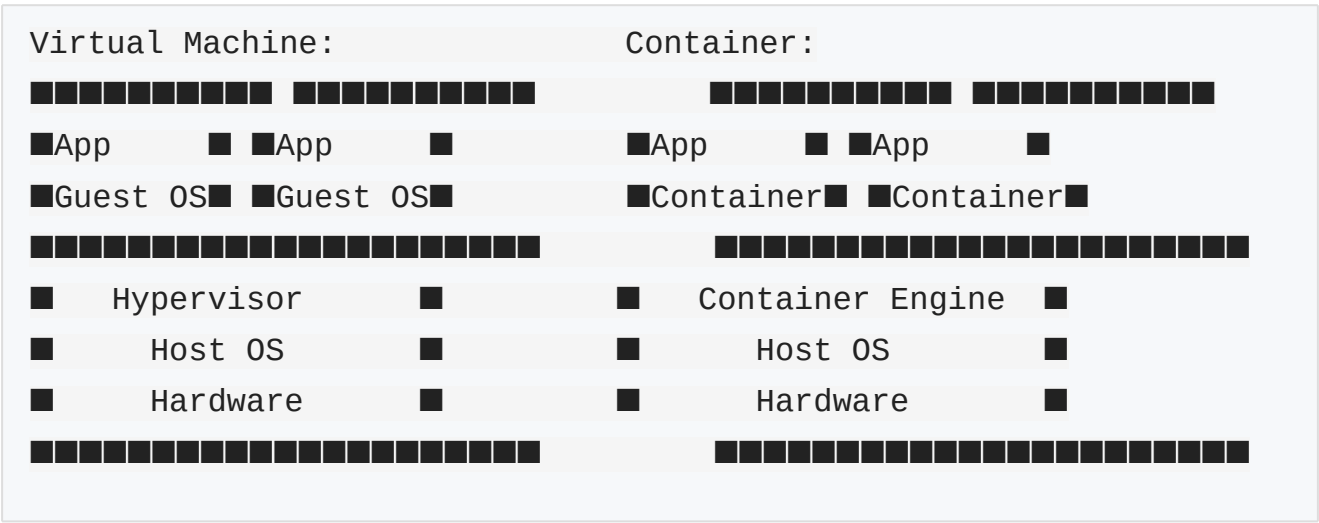
In One Line

Linux is a monolithic, modular OS with CFS scheduling, ext4 file system, and strong security support.

100. Virtual Machines vs Containers

Feature	Virtual Machine (VM)	Container
Isolation	Full OS per VM	Shared host OS kernel
OS	Each VM has own guest OS	Shares host OS; no guest OS
Size	GBs	MBs
Startup Time	Minutes	Seconds
Performance	Overhead of full OS	Near-native
Managed by	Hypervisor (VMM)	Container engine (Docker)
Security	Stronger isolation	Less isolated
Use case	Run different OSes, full isolation	Microservices, DevOps, scaling

Architecture



Key Technologies

- **VM:** VMware, VirtualBox, Hyper-V, KVM, Xen.
- **Container:** Docker, LXC, Podman; orchestrated by Kubernetes.
- **Container internals:** Linux **Namespaces** (isolation) + **cgroups** (resource limits) + **Union FS** (layered image).

In One Line

VMs virtualize hardware with full OS isolation; containers share the host kernel for lightweight, fast, portable isolation.

QUICK REVISION TABLE — ALL 100 TOPICS

#	Topic	Key Point
1	OS Basics	Intermediary between user and hardware
2	OS Functions	Process, memory, I/O, file, security, network, error, accounting
3	OS Types	Batch, multiprogramming, time-sharing, real-time, distributed, mobile
4	OS Services	Execution, I/O, file, communication, error, allocation, accounting, protection
5	Kernel	Core OS; runs in kernel mode; always in memory

6	User/Kernel Mode	Mode bit: 1=user, 0=kernel; switch via system call/interrupt
7	OS Structures	Monolithic, layered, microkernel, modular, hybrid
8	Monolithic vs Micro	Monolithic=fast,large; Micro=reliable,slow
9	System Calls	Controlled kernel entry; types: process, file, device, info, comm, protection
10	Interrupts	Hardware/software; saves state, runs ISR, restores state
11	Process	Program in execution; has code, stack, heap, data
12	Program vs Process	Passive vs active; disk vs memory
13	Process States	New, Ready, Running, Waiting, Terminated
14	State Diagram	Transitions: admitted, dispatch, interrupt, I/O wait/done, exit

15	PCB	PID, state, PC, registers, memory info, I/O info, accounting
16	Process Creation	Assign PID, allocate memory, init PCB, add to Ready queue
17	Process Termination	exit(); parent calls wait(); cascading termination
18	fork/exec/wait/exit	Create/replace/sync/terminate
19	Zombie	Finished but PCB remains; parent hasn't called wait()
20	Orphan	Parent terminated; adopted by init
21	Scheduling	Select process from Ready queue for CPU
22	Schedulers	Long-term=admit; short-term=dispatch; medium-term=swap
23	Dispatcher	Context switch + mode switch + jump; dispatch latency
24	Context Switch	Save PCB, load new PCB; pure overhead

25	Scheduling Criteria	Max: utilization, throughput; Min: TAT, WT, RT
26	FCFS	First arrival first; convoy effect; non-preemptive
27	SJF	Shortest burst first; optimal WT; needs burst time upfront
28	SRTF	Preemptive SJF; optimal; starvation risk
29	Priority	Higher priority first; starvation; aging
30	Round Robin	Fixed quantum; preemptive; fair; no starvation
31	Thread	Lightweight execution unit; shares process memory
32	Process vs Thread	Separate vs shared address space; heavy vs light
33	Multithreading Benefits	Responsiveness, resource sharing, economy, scalability

34	ULT vs KLT	ULT=fast,blocks all; KLT=true parallelism,slower
35	Many-to-One	All ULTs → 1 KLT; no parallelism
36	One-to-One	Each ULT → 1 KLT; true parallelism; Linux/Windows
37	Many-to-Many	Flexible mapping; most complex
38	IPC	Shared memory or message passing; pipes, sockets, signals
39	Shared Memory	Fastest IPC; needs synchronization
40	Message Passing	Safe, slower; no sync needed; good for distributed systems
41	Process Sync	Coordinate shared data access for consistency
42	Race Condition	Order-dependent wrong result from concurrent access
43	Critical Section	Code accessing shared resource; one at a time

44	CS Requirements	Mutual exclusion, progress, bounded waiting
45	Peterson's	Software 2-process solution; flag[] + turn
46	Atomic Ops	Execute as single unit; no interruption
47	Test-and-Set	Atomic read+set; implements spinlock
48	Compare-and-Swap	Atomic compare+update; lock-free algorithms
49	Semaphores	Integer + wait()/signal(); no busy-wait in blocking impl
50	Binary vs Counting	Binary=mutex(0/1); Counting=resource pool(0 to N)
51	Deadlock	Circular wait; all processes permanently blocked
52	Four Conditions	Mutual exclusion, hold-and-wait, no preemption, circular wait
53	Prevention	Eliminate one condition; low utilization

54	Avoidance	Banker's algorithm; stay in safe state
55	Detection	RAG cycle / detection algorithm; allow then detect
56	Recovery	Process termination or resource preemption
57	RAG	Request edge $P \rightarrow R$; assignment $R \rightarrow P$; cycle=deadlock
58	Safe vs Unsafe	Safe=sequence exists; unsafe=may deadlock
59	Banker's Algorithm	Grant only if safe sequence exists after allocation
60	Safe Sequence	Find P_i : $Need \leq Work$; update Work; repeat
61	Memory Management	Allocate, protect, reclaim RAM
62	Logical vs Physical	CPU generates logical; MMU translates to physical
63	Address Binding	Compile/load/execution time binding

64	MMU	Hardware; Physical = Logical + Base; limit check
65	Contiguous Alloc	Single block per process; fixed=internal frag; variable=external frag
66	First/Best/Worst Fit	First=fast; Best=least waste; Worst=worst
67	Fragmentation	Internal=inside partition; External=scattered holes
68	Compaction	Move processes; merge free holes; expensive
69	Swapping	Move process to disk; more processes than RAM
70	Paging	Pages→frames; no external frag; page table
71	Pages/Frames/PT	Page=logical block; frame=physical block; PT maps them
72	Address Translation	p=page#; d=offset; PT[p]=frame f; PA=f×pagesize+d
73	TLB	Hardware cache for PT entries; hit=fast; miss=slow

74	EAT	$\alpha(t+m)+(1-\alpha)(t+2m)$; hit ratio dominates
75	Multilevel Paging	Break PT into hierarchy; for large sparse address spaces
76	Segmentation	Variable-size logical segments; segment table; external frag
77	Virtual Memory	Run program larger than RAM; pages on disk
78	Demand Paging	Load pages only when needed; lazy loading
79	Page Fault	Access invalid page; trap; load from disk; restart
80	Page Replacement	FIFO/OPT/LRU/Clock; evict when no free frame
81	FIFO	Replace oldest; simple; Bélády's anomaly
82	Optimal	Replace future-unused; minimum faults; impractical
83	LRU	Replace least recently used; near-optimal; no anomaly

84	Second Chance	FIFO + reference bit; clock hand scans
85	Bélády's Anomaly	FIFO: more frames → more faults; LRU/OPT immune
86	Thrashing	More paging than execution; too few frames
87	Working-Set Model	Allocate frames = active page set; prevent thrashing
88	File Concept	Named data on disk; attributes, operations
89	File Access	Sequential, direct, indexed
90	File Allocation	Contiguous=fast; linked=flexible; indexed=balanced
91	Allocation Methods	Contiguous/linked/indexed; internal structure of file on disk
92	FCB/Inode	Per-file metadata; permissions, size, block pointers
93	I/O Management	Control devices via drivers, interrupts, DMA

94	Drivers/Controllers	Driver=software; controller=hardware; driver talks to controller
95	DMA	Device↔memory without CPU per byte; interrupt on done
96	Buffer/Cache/Spool	Buffer=speed match; cache=copy for speed; spool=queue for device
97	Disk Scheduling	Minimize seek time; FCFS/SSTF/SCAN/C-SCAN/LOOK/C-
98	All Disk Algorithms	FCFS=fair, SSTF=nearest, SCAN=elevator, C-LOOK=best
99	Linux Basics	Monolithic+modular; CFS; ext4/VFS; task_struct
100	VM vs Container	VM=full OS,heavy; Container=shared kernel,lightweight